

This lecture provides a comprehensive explanation of decorators in Python. Here's a structured summary:

1. Introduction to Decorators

- Decorators are used to enhance or modify the behavior of functions without altering their core logic.
- The concept is analogous to decorating a house: the original structure remains, but its appearance or functionality is improved.

2. Basic Decorator Example

- A simple `greet` function is defined to illustrate the basic concept.
- A `my_deco_function` is created to act as a decorator. It takes a function as input and defines an inner `wrapper` function.
- The `wrapper` function adds functionality before and after the original function's execution (e.g., printing messages).
- The `@my_deco_function` syntax is used to apply the decorator to the `greet` function.
- When `greet` is called, the `wrapper` function executes, adding the pre- and post-execution steps.

3. Creating a Practical Decorator: Time Calculator

- A more useful decorator, `time_calculator_deco`, is developed to measure the execution time of a function.
- The decorator uses the `time` module to record the start and end times of the function's execution.
- It calculates and prints the total time taken.
- This decorator is designed to be generic, accepting functions with any number of arguments using `*args` and `**kwargs`.

4. Applying the Time Calculator Decorator

- A `print_list` function is created to filter even numbers from a list and simulate a time-consuming operation using `time.sleep()`.
- The `@time_calculator_deco` is applied to `print_list`.
- The lecture demonstrates how the decorator automatically measures and prints the execution time of `print_list`.

5. Stacking Decorators

- The lecture explains how to use multiple decorators on a single function.
- It demonstrates applying both `time_calculator_deco` and `my_deco_function` to `print_list`.
- The order in which decorators are stacked matters, as they are applied from top to bottom.
- Adjustments to the decorator functions (using `*args` and `**kwargs` in the inner function) are necessary to ensure compatibility when stacking.

6. Key Takeaways

- Decorators provide a clean and reusable way to add functionality to functions.
- They promote code modularity and reduce redundancy.
- Decorators can be stacked to apply multiple enhancements.
- Understanding `*args` and `**kwargs` is crucial for creating generic decorators.
- The lecture emphasizes the power and flexibility of decorators in Python for tasks such as logging, timing, and authentication.
-